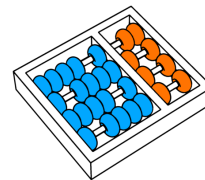




RELATÓRIO DE PROJETO FINAL
Curso de Android e Internet das Coisas (IoT)



VanGuard Kids

Monitoramento de ocupação para segurança no transporte escolar

Autor(es): Isaac Vinícius e Samuel Henrique

Sumário

Sumário	2
1. Resumo do Projeto	2
2. Definição do Problema e Visão de Sistema	3
2.1. O problema	3
2.2. A solução proposta	3
2.3. Mapeamento dos pilares do curso	3
3. Arquitetura do Sistema	4
3.1. Diagrama da arquitetura	5
3.2. Hardware e Sistema Operacional	5
3.3. Aplicativo Android	7
3.4. Conectividade e Protocolos	7
3.5. Plataforma de IoT	9
4. Desenvolvimento e Implementação	9
4.1. Etapas de desenvolvimento	9
4.2. Principais desafios e soluções	11
4.3. Trechos de código relevantes	11
5. Resultados e Testes	15
5.1. Demonstração do protótipo	15
5.2. Testes realizados	15
6. Conclusão	18
7. Referências	19
Anexos	21
Anexo A — Diagramas de Arquitetura	21
Anexo B — Fluxogramas	23
Anexo C — Protótipo	25
Anexo D — Vídeos de Demonstração	30
Anexo E — Material Complementar	31

1. Resumo do Projeto

O VanGuard Kids é um sistema IoT focado em mitigar o problema do esquecimento de crianças no interior de veículos de transporte escolar, uma situação que representa grave risco à segurança infantil. A solução atua como uma camada de segurança preventiva, utilizando um microcontrolador ESP32 equipado com sensores de peso para monitorar a ocupação dos assentos e transmitir esses dados localmente via Wi-Fi, por meio do protocolo MQTT. Para a interação do motorista, foi desenvolvida uma aplicação Android embarcada que exibe, em tempo real, um mapa de assentos e um contador de ocupação. Além de conter um mapa estático para simular a rota da van, o aplicativo bloqueia logicamente o encerramento da viagem caso alguma criança ainda esteja a bordo, acionando um alerta crítico. O sistema também integra a plataforma Node-RED para a exibição de informações em dashboards. O resultado é um protótipo funcional que une hardware, software e conectividade para prevenir ativamente possíveis incidentes.

2. Definição do Problema e Visão de Sistema

2.1. O problema

No transporte escolar, há risco de crianças serem esquecidas dentro do veículo após o encerramento da rota, especialmente quando estão dormindo, em assentos menos visíveis ou não são percebidas pelo motorista durante o desembarque. Esse tipo de situação pode representar risco grave à segurança infantil.

O VanGuard Kids foi pensado durante a leitura de algumas reportagens (Anexo E) sobre casos de crianças que foram esquecidas dentro de transportes escolares. Existem muitos casos no Brasil e ao redor do mundo de crianças esquecidas dentro de veículos escolares, e em alguns casos mais sérios vindo até a falência.

2.2. A solução proposta

O VanGuard Kids atua como uma camada adicional de apoio ao motorista através de um sistema microcontrolado para leitura do estado dos assentos (ocupado ou não) e de um aplicativo Android com interface gráfica para visualização e controle do motorista. O sistema fornece:

- Monitoramento da ocupação dos assentos;
- Visualização em tempo real do estado dos assentos;
- Gerenciamento de rota (início e fim) e quantidade de paradas;
- Alertas em caso de presença detectada após o fim da rota;
- Registro e encaminhamento dos dados para uma plataforma por meio de uma arquitetura IoT e protocolo MQTT;
- Histórico e rastreabilidade dos dados.

2.3. Mapeamento dos pilares do curso

A solução conecta diferentes pilares que foram abordados durante o curso o que torna o projeto multidisciplinar. A Tabela 1 abaixo aborda cada pilar e como o projeto o conecta.

Tabela 1 – Pilares e conexão com o projeto

Pilar do curso	Como o projeto utiliza este pilar
Hardware / SO (Android Embarcado e Conceitos de SO)	<i>O projeto utiliza uma Raspberry Pi 5B com Android Embarcado como unidade central do sistema, funcionando como uma central multimídia veicular para exibir o estado dos assentos ao motorista. O ESP32 junto com o FreeRTOS atua como dispositivo embarcado de aquisição, realizando a leitura dos sensores de pressão instalados nos assentos. O firmware possui duas tasks, uma que realiza a leitura dos assentos e outra que envia os dados até o broker. A comunicação entre elas ocorre através de uma fila.</i>
App (Android Básico e Ferramentas)	<i>Foi desenvolvido um aplicativo Android para exibir ao motorista o status dos assentos da van escolar, indicando quais estão livres ou ocupados. A aplicação também apresenta alertas visuais quando há risco de permanência de criança no veículo. O app foi estruturado com telas de visualização, componentes de interface e lógica de atualização em tempo real a partir dos dados recebidos via MQTT, seguindo a arquitetura MVVM. O desenvolvimento utiliza conceitos de Android, Kotlin, ciclo de vida da aplicação, layouts, depuração com Android Studio, ADB e Logcat.</i>
Conectividade (Protocolos de Redes e Tecnologias Sem Fio)	<i>A conectividade do sistema é baseada em Wi-Fi e MQTT. O ESP32 conecta-se à rede Wi-Fi local e envia payloads em formato JSON via MQTT para o broker. A Raspberry Pi 5B, executando Android, conecta-se à mesma rede Wi-Fi, inscreve-se nos tópicos MQTT correspondentes e recebe os dados de ocupação dos assentos. Dessa forma, o projeto aplica conceitos de rede local, endereçamento IP, comunicação entre dispositivos e protocolo de aplicação MQTT em um cenário IoT.</i>
Plataforma (Plataformas de IoT, se necessário)	<i>Como plataforma IoT local, o projeto utiliza o Node-RED, responsável por concentrar os dados recebidos via MQTT, organizar os fluxos, montar dashboards e gerar alertas. O Mosquitto foi utilizado como broker MQTT principal, executado localmente em um computador Linux. Essa arquitetura permite visualizar os dados do sistema, acompanhar eventos de ocupação e validar o funcionamento da comunicação entre sensores, ESP32, broker, Node-RED e aplicativo Android.</i>

Fonte: elaborado pelo autor (2026).

3. Arquitetura do Sistema

A arquitetura integra hardware embarcado, aplicativo Android, conectividade sem fio e plataforma de visualização de dados. O sistema é composto por quatro módulos principais:

1. **Firmware:** responsável pela aquisição dos dados dos sensores e pelo envio das informações ao broker MQTT.
2. **Broker MQTT:** responsável por intermediar a comunicação entre os dispositivos publicadores e consumidores.
3. **Node-RED:** responsável por concentrar os dados recebidos via MQTT, montar dashboards, gerar alertas e manter um histórico visual dos eventos.

4. **Aplicativo Android:** responsável por exibir ao motorista o estado dos assentos, indicando quais estão livres ou ocupados, além de apresentar alertas visuais em caso de risco.

A comunicação entre os módulos ocorre principalmente por meio do protocolo MQTT, utilizado para publicar e consumir os dados de ocupação dos assentos. O ESP32 publica as mensagens contendo o estado dos sensores, enquanto o Node-RED e o aplicativo Android se inscrevem nos tópicos MQTT para receber e processar os dados em tempo real.

3.1. Diagrama da arquitetura

A Figura 1, apresentada no **Anexo A**, ilustra a arquitetura geral do sistema, o papel de cada componente e o fluxo de comunicação entre eles.

O firmware executado no ESP32 é responsável por coletar os dados dos sensores de pressão conectados aos assentos. Após a leitura, o ESP32 monta mensagens em formato JSON e publica essas informações no broker MQTT por meio da rede Wi-Fi local.

O broker MQTT, executado localmente em um computador Linux, atua como intermediário da comunicação. A partir dele, tanto o Node-RED quanto a aplicação Android podem se inscrever nos tópicos e receber os dados enviados pelo ESP32.

O Node-RED utiliza esses dados para montar dashboards, organizar o histórico e gerar alertas. Já o aplicativo Android, executado na Raspberry Pi 5B com Android Embarcado, funciona como uma interface de apoio ao motorista, exibindo de forma visual o estado de cada assento da van escolar e enviando informações sobre a rota até o broker.

De forma geral, o fluxo de dados pode ser descrito da seguinte maneira:

1. Os sensores de peso detectam a ocupação dos assentos.
2. O ESP32 realiza a leitura dos sensores.
3. O firmware monta um payload JSON com o estado dos assentos.
4. O ESP32 publica os dados no broker MQTT.
5. O broker MQTT distribui os dados para os consumidores inscritos.
6. O Node-RED recebe os dados e atualiza o dashboard.
7. O aplicativo Android recebe os dados e atualiza a interface exibida ao motorista.
8. Caso algum assento permaneça ocupado em uma situação de risco, o sistema exibe alertas visuais no dashboard e na aplicação Android.

3.2. Hardware e Sistema Operacional

O principal dispositivo de aquisição utilizado no projeto é o **ESP32**, responsável pela leitura dos sensores de peso instalados nos assentos. Cada sensor representa um assento e permite identificar se ele está livre ou ocupado. No desenvolvimento do firmware, o hardware foi completamente simulado na plataforma **Wokwi** [13][14], conforme descrito na Tabela 2.

Tabela 2 – Hardware utilizado

Hardware	Descrição
ESP32 DevKit C	<i>Microcontrolador versátil e amplamente utilizado em projetos IoT. Possui integração nativa com o FreeRTOS e conectividade Wi-Fi, o que facilita a implementação da comunicação via MQTT. Além disso, dispõe de múltiplos canais ADC, utilizados para a leitura dos sensores de peso simulados.</i>
Potenciômetros	<i>Os sensores de peso foram simulados por meio de quatro potenciômetros, que utilizam o mesmo princípio de aquisição analógica via ADC. Essa escolha foi adotada em razão do custo dos sensores reais, do prazo de entrega e da maior facilidade de reprodução e teste por ambos os integrantes do grupo.</i>

Fonte: elaborado pelo autor (2026).

Para o firmware, foi projetada uma arquitetura organizada em três camadas principais:

- **Hardware:** composta pelo ESP32 e pelos potenciômetros utilizados para simular os sensores de peso dos assentos.
- **ESP-IDF:** camada composta pelos componentes de software responsáveis pela configuração dos periféricos, comunicação de rede e sincronização de tempo.
- **FreeRTOS:** camada responsável pela execução concorrente das tarefas do sistema, utilizando duas tasks principais: vTaskSeat e vTaskMqtt.

A arquitetura do firmware, ilustrada na Figura 2 do Anexo A, foi projetada com o objetivo de separar responsabilidades e facilitar a manutenção do sistema. Para o desenvolvimento, foi utilizado o **ESP-IDF**, SDK oficial da Espressif para programação do ESP32.

Na camada de ESP-IDF, foram desenvolvidos três componentes principais:

- **adc_manager:** responsável por configurar, gerenciar e realizar as leituras dos canais ADC aos quais os sensores dos assentos estão conectados [3].
- **mqtt_manager:** responsável por gerenciar a conexão Wi-Fi, a conexão com o broker MQTT, a publicação de mensagens e a subscrição em tópicos [1][2].
- **time_manager:** responsável por inicializar o SNTP e sincronizar o relógio do dispositivo por meio da rede Wi-Fi [13].

Na camada de Sistema Operacional, o projeto utiliza o **FreeRTOS**, que permite a execução concorrente de tarefas no ESP32. Esse recurso foi utilizado para separar a aquisição dos dados dos sensores da comunicação MQTT, evitando que uma atividade bloqueie a outra. Além disso, foi utilizada uma fila de mensagens para permitir a comunicação segura entre as tasks.

Foram desenvolvidas duas tasks principais:

- **vTaskSeat:** responsável por ler periodicamente os sensores, identificar o estado dos assentos, montar a mensagem intermediária e enviá-la para uma fila. A task é executada a cada segundo, porém uma nova mensagem é preparada apenas quando há mudança no estado de algum assento.

- **vTaskMqtt**: responsável por enviar mensagens de heartbeat a cada 30 segundos e consumir a fila de mensagens gerada pela vTaskSeat. Essa task verifica a fila a cada segundo, formata os dados em JSON e publica as mensagens nos tópicos MQTT correspondentes.

A utilização de tasks e filas evidencia a aplicação de conceitos de sistemas operacionais embarcados, como concorrência, comunicação entre processos leves, sincronização e gerenciamento de recursos. Dessa forma, o firmware consegue realizar a leitura dos sensores, manter a conexão de rede e publicar mensagens MQTT de maneira organizada e não bloqueante.

O Anexo B apresenta os fluxogramas das tasks **vTaskSeat** e **vTaskMqtt**, ilustrados nas Figuras 5 e 6, respectivamente. Além disso, a Figura 4 apresenta a rotina de inicialização do firmware.

3.3. Aplicativo Android

A aplicação Android foi desenvolvida utilizando a linguagem Kotlin e o ambiente de desenvolvimento Android Studio. Essa abordagem favorece a modularidade, torna as transições mais fluidas e reduz o consumo de memória e de recursos do sistema, conforme recomendado pela documentação oficial do Android [5].

As principais telas incluem: o **MapFragment** (para funcionalidades de navegação), o **SeatingFragment** (que renderiza a grade dinâmica de assentos da van) e o **SeatingStatusFragment** (que exibe os contadores em tempo real de ocupação). A principal funcionalidade de segurança desenvolvida é o **SafetyBottomSheet**, uma gaveta inferior de alerta crítico ativada caso o motorista tente encerrar a rota com crianças a bordo. As telas implementadas podem ser vistas no Anexo C.

A comunicação IoT é realizada por meio da biblioteca Eclipse Paho [4], utilizada como cliente MQTT para receber os dados dos sensores e enviar ao broker as atualizações de rota do motorista. Sua adoção se deve ao fato de ser uma solução amplamente consolidada em aplicações Android, oferecendo estabilidade na manutenção de conexões contínuas, suporte à reconexão automática e gerenciamento eficiente de operações assíncronas por meio de callbacks. Além disso, o aplicativo utiliza recursos modernos do Android, como StateFlow e Coroutines, para garantir a reatividade da interface e manter um fluxo unidirecional de dados [6].

Além da comunicação de rede via MQTT para a leitura dos assentos, o aplicativo interage diretamente com o hardware físico do dispositivo Android por meio do AIDL (Android Interface Definition Language). Conforme ilustrado na arquitetura do sistema (Figuras 1 e 3), a aplicação utiliza a interface IMag.aidl para estabelecer uma comunicação interprocessos (IPC via Binder) com o serviço de baixo nível do sistema operacional (MagService e Mag HAL). Essa integração permite que a classe MagDataSource acesse os dados diretos do magnetômetro local, fundamentais para o processamento de azimuth e orientação espacial exibidos em tempo real na interface do MapFragment.

3.4. Conectividade e Protocolos

A conectividade do sistema é baseada em **Wi-Fi** e **MQTT**. Essa escolha foi feita porque o Wi-Fi já está disponível tanto no ESP32 quanto na Raspberry Pi 5B, permitindo uma comunicação local simples e adequada para o protótipo. O MQTT foi escolhido por ser um protocolo leve, baseado no modelo publish/subscribe, adequado para aplicações IoT em que dispositivos

precisam enviar e receber mensagens de forma desacoplada, e por ter sido bastante abordado durante as aulas com exemplos práticos no Android.

A comunicação geral ocorre da seguinte forma:

- **ESP32 → Broker MQTT:** envio dos dados dos sensores via Wi-Fi e MQTT.
- **Broker MQTT → Aplicativo Android:** entrega dos dados de ocupação para exibição ao motorista.
- **Broker MQTT → Node-RED:** entrega dos dados para dashboard, histórico e alertas.
- **Node-RED → Dashboard:** visualização dos dados em interface web local.

Os tópicos foram estruturados da seguinte forma:

- **vanguard-kids/seats/seat-XX:** tópico para informações dos assentos, sendo XX o número do assento, 01, 02, 03 e 04.
- **vanguard-kids/heartbeat:** tópico com informações do heartbeat enviado pela ESP32.
- **vanguard-kids/route:** tópico com informações sobre a rota iniciada pelo motorista enviado pela aplicação Android.

Os dados trafegam em formato JSON, facilitando a interpretação das mensagens pelos diferentes módulos do sistema.

Exemplo de payload do heartbeat:

```
{
  "client_id": "<client_id>",
  "ts": "<epoch>",
  "seats": 4
}
```

Exemplo de payload de assento:

```
{
  "client_id": "<client_id>",
  "seat": "seat-01",
  "adc_raw": 1234,
  "voltage": 1.22,
  "weight_g": 1520,
  "occupied": true,
  "ts": 1699999999
}
```

Exemplo de payload de rota:

```
{
  "name": "Rota da manhã",
  "stops": "4",
  "seats_occupied": 2,
  "ts": 1782486992274,
}
```



```
"state": "In Progress",  
"block_alert": true  
}
```

3.5. Plataforma de IoT

Como plataforma IoT local, o projeto utiliza o Node-RED, integrado ao broker MQTT Mosquitto. Essa escolha permite criar uma solução funcional sem depender obrigatoriamente de serviços em nuvem, facilitando a demonstração em ambiente controlado.

O Node-RED é uma ferramenta open-source focado em programação low-code para aplicações orientadas a eventos e permite ampla customização. É utilizado para concentrar, processar e visualizar os dados recebidos. Por meio de fluxos visuais, é possível organizar as mensagens MQTT, montar dashboards, gerar alertas e acompanhar o histórico de ocupação dos assentos.

As principais funções da plataforma IoT no projeto são:

- receber os dados publicados pelo ESP32;
- concentrar as informações de ocupação dos assentos;
- permitir a visualização em dashboard web;
- gerar alertas com base no estado dos assentos;
- auxiliar na depuração e validação do sistema;

Dessa forma, a plataforma IoT complementa o aplicativo Android. Enquanto o aplicativo é voltado ao uso direto pelo motorista dentro do veículo, o Node-RED funciona como ambiente de monitoramento, validação e visualização dos dados do sistema.

4. Desenvolvimento e Implementação

O desenvolvimento do VanGuard Kids foi organizado em etapas incrementais, partindo da definição da arquitetura do sistema até a integração entre firmware, broker MQTT, Node-RED e aplicativo Android. A implementação priorizou a construção de um protótipo funcional e reproduzível, utilizando simulação no Wokwi para a camada de hardware e execução local dos serviços de comunicação e visualização. Do lado da aplicação Android, a implementação focou em criar uma interface responsiva para o motorista. A integração dos dados de telemetria recebidos via MQTT com os componentes visuais foi feita de forma fluida utilizando a arquitetura MVVM, assegurando que o aplicativo pudesse ser executado no Android Embarcado na Raspberry Pi 5 sem comprometer a estabilidade do sistema ou causar gargalos de processamento.

Como resultado, todo código e artefato produzido pode ser encontrado no repositório do Github: [vanguard-kids](#).

4.1. Etapas de desenvolvimento

A primeira etapa do projeto consistiu na definição do problema que a solução buscava resolver. Após algumas pesquisas, foram encontrados diversos casos relatados em reportagens envolvendo crianças esquecidas dentro de veículos de transporte, não apenas em transportes

escolares. O Anexo E apresenta algumas reportagens de casos brasileiros que serviram como motivação para a criação deste projeto.

A segunda etapa envolveu a definição dos principais elementos técnicos da solução, como o sensor a ser utilizado, a arquitetura do firmware, a arquitetura do aplicativo Android, o fluxo de comunicação entre os módulos e os protocolos adotados.

Como resultado dessa etapa, as Figuras 1, 2 e 3, apresentadas no Anexo A, ilustram as arquiteturas definidas para o sistema. No firmware, foi priorizada uma arquitetura simples, organizada em camadas e alinhada, sempre que possível, ao padrão de desenvolvimento adotado pelo ESP-IDF e pela Espressif. Além disso, optou-se por simular toda a camada de hardware utilizando a plataforma Wokwi.

Para a aplicação Android, foi escolhida a arquitetura Model-View-ViewModel (MVVM), composto por três camadas principais: a camada de Apresentação (View), contendo as Activities e Fragments responsáveis pela interface gráfica; a camada Lógica (ViewModel), que processa e mantém o estado dos dados sobrevivendo às mudanças de ciclo de vida; e a camada de Dados (Model/Repository), atuando como a única fonte da verdade (*Single Source of Truth*) [7], [8], para consolidar as leituras do hardware e as mensagens MQTT.

A terceira etapa foi composta pelo desenvolvimento do firmware e do aplicativo Android. De forma paralela, com cada integrante da equipe atuou em uma frente específica: Isaac ficou responsável pela aplicação Android, enquanto Samuel ficou responsável pela concepção e desenvolvimento do firmware.

No desenvolvimento do firmware, inicialmente foi criado um projeto no Wokwi para a montagem do circuito elétrico envolvendo o ESP32 e os potenciômetros utilizados para simular os sensores de peso. Em seguida, foi criado um projeto baseado em ESP-IDF, no qual foram desenvolvidos os componentes `mqtt_manager`, `adc_manager` e `time_manager`. Posteriormente, foram implementadas as tarefas do FreeRTOS responsáveis pela aquisição dos dados dos sensores e pelo envio das mensagens MQTT. Também foi definida a estrutura de tópicos utilizada para publicação e recebimento das mensagens entre o firmware, o broker, o Node-RED e o aplicativo Android.

No desenvolvimento do aplicativo Android, a construção ocorreu em três fases principais. Inicialmente, estruturou-se o projeto no Android Studio com a configuração das dependências (como a biblioteca Eclipse Paho para o MQTT) e a proteção das credenciais do Broker utilizando o arquivo **local.properties** e a definição da interface de baixo nível `IMag.aidl` para viabilizar a comunicação com o HAL do magnetômetro via Binder IPC. Em seguida, as interfaces modulares (Fragments) foram implementadas para compor a tela principal, exibindo a navegação, a grade dinâmica de assentos e os contadores de status. Na última fase, desenvolveu-se a lógica de negócio por meio de componentes reativos (StateFlow e Coroutines), estabelecendo o "laço de segurança" (ilustrado no fluxo do usuário da Figura 7): uma regra que impede o motorista de finalizar a rota caso os dados recebidos via MQTT indiquem presença nos assentos, levantando a tela de alerta crítico `SafetyBottomSheet`.

A quarta etapa foi composta pela configuração do broker MQTT Mosquitto, incluindo a criação de usuário e senha para autenticação dos dispositivos. Também foram realizados testes com os comandos `mosquitto_pub` e `mosquitto_sub`, permitindo validar o envio e o recebimento dos payloads publicados pelo ESP32.

Na quinta etapa, foi desenvolvido o fluxo no Node-RED, contemplando a integração com a comunicação MQTT, a montagem do dashboard e a criação dos cards para exibição dos

assentos livres e ocupados, das informações da rota e do heartbeat do sistema. Também foi implementado o armazenamento do histórico de ocupação dos assentos e das informações da rota por meio de tabelas. Além disso, foram desenvolvidos alertas por notificações e classes CSS para customização visual dos elementos do dashboard.

A sexta etapa consistiu na integração de todas as partes do projeto. Nessa fase, o firmware passou a enviar os dados dos sensores, o aplicativo Android passou a recebê-los e a executar a lógica de gerenciamento da rota controlada pelo motorista, enquanto o dashboard no Node-RED passou a monitorar o sistema por meio de gráficos, cards e tabelas com as informações trafegadas. Também foram realizados testes para validar os alertas gerados em diferentes situações.

Por fim, a última etapa consistiu na documentação do projeto, com a criação de arquivos README para cada uma das partes desenvolvidas, incluindo firmware, aplicativo Android e Node-RED.

4.2. Principais desafios e soluções

Tabela 3 – Desafios e soluções adotadas

Desafio	Impacto no projeto	Solução adotada
Aquisição dos sensores reais de peso	Dificuldade na montagem física e na reprodução do protótipo	Os sensores foram simulados com potenciômetros no Wokwi, utilizando leitura ADC equivalente
Customização visual do dashboard no Node-RED	O Node-RED permite a personalização dos componentes principalmente por meio de CSS. Como a equipe não possuía domínio avançado nessa tecnologia, houve dificuldade inicial para ajustar a aparência dos cards, alertas e demais elementos visuais.	Foi utilizada IA generativa como ferramenta de apoio para auxiliar na criação das classes CSS utilizadas no dashboard. As sugestões geradas foram revisadas, ajustadas e testadas pela equipe até atenderem às necessidades visuais do projeto.

Fonte: elaborado pelo autor (2026)

4.3. Trechos de código relevantes

Código-fonte 1 – Tarefa de aquisição de dados

```
static void vTaskSeats(void* pvParameters) {
    UNUSED(pvParameters);

    SeatMessage message;

    while (1) {
        for (int i = 0; i < SEAT_COUNT; i++) {
```

```

Seat* seat = &seats[i];

if (!update_seat(seat)) {
    continue;
}

bool should_publish = !seat->initialized || (seat->occupied != seat->last_occupied);

if (!should_publish) {
    continue;
}

seat->last_occupied = seat->occupied;
seat->initialized = true;

// Prepare the message to send to the queue
message.id = seat->id;
message.adc_raw = seat->adc_raw;
message.voltage = seat->voltage;
message.weight_grams = seat->weight_grams;
message.occupied = seat->occupied;
message.ts = get_epoch();

strncpy(message.name, seat->name, sizeof(message.name));
message.name[sizeof(message.name) - 1] = '\0';

if (xQueueSend(xSeatQueue, &message, pdMS_TO_TICKS(100)) != pdPASS) {
    ESP_LOGW(TAG, "Fila cheia — mensagem descartada: %s", message.name);
} else {
    ESP_LOGI(TAG, "Mudança enfileirada: %s → %s", message.name, message.occupied
? "OCUPADO" : "LIVRE");
}
}

vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

Fonte: elaborado pelo autor (2026)

Código-fonte 2 – Tarefa de envio de mensagens MQTT

```

static void vTaskMQTT(void* pvParameters) {
    UNUSED(pvParameters);

    SeatMessage msg;
    char topic[64];
    char payload[256];

    // Initialize last_heartbeat to ensure the first heartbeat is sent immediately

```

```

    TickType_t last_heartbeat = xTaskGetTickCount() -
pdMS_TO_TICKS(HEARTBEAT_INTERVAL_MS);

while (1) {
    TickType_t now = xTaskGetTickCount();
    bool heartbeat_due = (now - last_heartbeat) >= pdMS_TO_TICKS(HEARTBEAT_INTERVAL_MS);

    if (heartbeat_due) {
        if (mqtt_is_connected()) {
            snprintf(payload, sizeof(payload),
                "{\"client_id\":\"%s\",\"ts\":\"%\" PRId64 \"\", \"seats\":%d}\",
                CONFIG_MQTT_CLIENT_ID,
                get_epoch(),
                SEAT_COUNT
            );

            mqtt_publish(MQTT_TOPIC_HEARTBEAT, payload);
            ESP_LOGI(TAG, "Heartbeat publicado");
        } else {
            // Broker offline. Wait for connection before sending heartbeat
            wait_mqtt_connection("heartbeat");
        }

        last_heartbeat = now;
    }

    /*
    * Wait for a seat message from the queue with a timeout to allow heartbeat checks.
    * If no message is received within the timeout, the loop continues to check for heartbeat.
    */
    if (xQueueReceive(xSeatQueue, &msg, pdMS_TO_TICKS(1000)) != pdPASS) {
        continue; // No message received, continue to next iteration to check heartbeat
    }

    // Prepare MQTT topic and payload for the seat message
    snprintf(topic, sizeof(topic), MQTT_TOPIC_SEAT_FMT, msg.name);

    snprintf(
        payload,
        sizeof(payload),
        "{\"client_id\":\"%s\",\"seat\":\"%s\",\"adc_raw\":%d,\"voltage\":%.2f,\"weight_g\":%.0f,\"occupied\":%s,\"ts\":\"%\" PRId64}\",
        CONFIG_MQTT_CLIENT_ID,
        msg.name,
        msg.adc_raw,
        msg.voltage,
        msg.weight_grams,
        msg.occupied ? "true" : "false",
        msg.ts
    );

```

```

);

if (mqtt_is_connected()) {
    mqtt_publish(topic, payload);
    ESP_LOGI(TAG, "Publicado → %s", topic);
} else {
    // MQTT offline — re-enqueue the message for retry
    ESP_LOGW(TAG, "MQTT offline. Re-enfileirando: %s", msg.name);

    if (xQueueSendToFront(xSeatQueue, &msg, 0) != pdPASS) {
        ESP_LOGE(TAG, "Fila cheia. Mensagem perdida: %s", msg.name);
    }

    wait_mqtt_connection("publish");
}
}
}
}

```

Fonte: elaborado pelo autor (2026)

Código-fonte 3 – Lógica de validação e laço de segurança (Aplicativo Android)

```

// Trecho extraído de SeatingViewModel.kt
fun handleRouteAction() {
    viewModelScope.launch {
        val currentState = routeState.value.state

        if (currentState == "In Progress") {
            val currentSeats = uiState.value.seats
            // Filtra e extrai a lista de assentos ocupados a partir do estado reativo
            val occupiedSeatsList = currentSeats.values.filter { it.isOccupied == true }

            if (occupiedSeatsList.isNotEmpty()) {
                // Laço de segurança: Bloqueia o encerramento se houver crianças
                val errorMsg = "Bloqueado: Criança a bordo!"
                repository.updateRouteState(errorMsg, nextState = "In Progress")
                repository.dispatchRouteTelemetry(isBlocked = true)

                // Emite o evento (SharedFlow) para o Fragment exibir o alerta crítico na tela
                _uiEvent.emit(UiEvent.ShowSecurityDialog(occupiedSeatsList))
            } else {
                // Encerra a rota com sucesso caso todos os assentos estejam livres
                val successMsg = "Rota finalizada"
                repository.updateRouteState(successMsg, nextState = "Completed")
                repository.dispatchRouteTelemetry(isBlocked = false)
                _uiEvent.emit(UiEvent.ShowSuccessToast)
            }
        }
    }
}
}
}
}

```

Fonte: elaborado pelo autor (2026)

O trecho de código acima, presente no SeatingViewModel, demonstra a lógica central do aplicativo Android que atende ao principal objetivo do projeto: evitar o esquecimento de crianças. Através do monitoramento passivo do estado da van (`uiState.value.seats`), o sistema atua de forma preventiva, exibindo o alerta visual (`ShowSecurityDialog`) e travando a conclusão da rota caso a filtragem identifique qualquer assento ainda ocupado durante a tentativa de encerramento pelo motorista.

5. Resultados e Testes

O protótipo apresentou resultados satisfatórios nos testes realizados. Todos os resultados estão descritos abaixo, as evidências presentes no Anexo C, bem como imagens que mostram o funcionamento do protótipo. Além disso, o vídeo de demonstração está presente no Anexo D.

A reprodução desse projeto pode ser executada seguindo o passo-a-passo descrito nos arquivos READMEs.

5.1. Demonstração do protótipo

O vídeo apresentado no Anexo C percorre todas as frentes do projeto e demonstra o funcionamento geral do VanGuard Kids.

Com a simulação inicialmente parada, o vídeo apresenta uma breve explicação sobre o objetivo do projeto, mostrando o circuito eletrônico montado no Wokwi e a tela do aplicativo Android espelhada por meio da ferramenta `scrcpy` [15]. Após essa introdução, a demonstração passa para a plataforma Node-RED, destacando o dashboard desenvolvido e customizado para o projeto. Nessa etapa, são apresentadas as principais abas e páginas do dashboard, explicando a função de cada uma delas no monitoramento da ocupação dos assentos, das rotas e do estado do sistema.

Em seguida, a simulação é iniciada e passa a ser demonstrada a comunicação entre o firmware, o broker MQTT, o Node-RED e o aplicativo Android. O vídeo mostra os dados sendo transmitidos pelo ESP32 e refletidos tanto no dashboard quanto na aplicação Android. Também é apresentada a alteração visual das telas conforme os estados dos assentos mudam, além da funcionalidade de seleção e acompanhamento das rotas.

Por fim, são demonstrados os dois principais cenários de alerta implementados no sistema:

- O ESP32 fica offline enquanto ainda reportava assentos ocupados;
- Há uma tentativa de encerramento de rota enquanto pelo menos um assento permanece ocupado.

Em ambos os casos, os alertas são exibidos no Node-RED por meio de notificações e cards visuais no dashboard. Além disso, também é apresentada a notificação gerada na bandeja do sistema. O fluxo de usuário apresentado no Anexo B, Figura 7, ilustra o comportamento esperado durante o uso do sistema e complementa os cenários demonstrados no vídeo.

5.2. Testes realizados

Para o projeto, foram realizados os testes abaixo.

Teste 1. Comunicação entre ESP32 e Broker MQTT	
Objetivo	Validar a comunicação entre ESP32 e Broker MQTT.
Passos	1. Iniciar a simulação da ESP32. 2. Enviar dados para os tópicos <code>vanguard-kids/heartbeat</code> e <code>vanguard-kids/seats/seat-01/02/03/04</code> . 3. Mudar o estado do assento através do potenciômetro.
Resultado esperado	1. Dados do heartbeat devem chegar a cada 30 segundos. 2. Dados dos assentos devem chegar sempre que uma mudança de estado acontecer.
Resultado obtido	O resultado obtido foi o esperado. Para avaliar, foi utilizado o comando <i>mosquitto_sub</i> para se inscrever nos tópicos e observar a chegada dos dados corretamente. Evidência no Anexo C, Figuras 9 e 10.
Status	Aprovado.

Teste 2. Integração com Node-RED	
Objetivo	Validar se o dashboard recebe e exibe os dados corretamente.
Passos	1. Iniciar a simulação da ESP32. 2. Enviar dados para os tópicos <code>vanguard-kids/heartbeat</code> e <code>vanguard-kids/seats/seat-01/02/03/04</code> . 3. Mudar o estado do assento através do potenciômetro.
Resultado esperado	1. Dados do heartbeat são mostrados no dashboard de heartbeat. 2. Dados dos assentos são mostrados no mapa de assentos. 3. Dados são mostrados nas tabelas de histórico.
Resultado obtido	Cards e tabelas foram atualizados corretamente. Evidência no Anexo C, Figuras 11, 12 e 13.
Status	Aprovado.

Teste 3. Integração com aplicativo Android	
Objetivo	Validar se a aplicação Android recebe os dados via MQTT.
Passos	1. Iniciar a simulação da ESP32. 2. Enviar dados para os tópicos <code>vanguard-kids/seats/seat-01/02/03/04</code> . 3. Mudar o estado do assento através do potenciômetro.

Resultado esperado	1. A interface deve exibir corretamente os assentos livres e ocupados.
Resultado obtido	A interface foi atualizada corretamente com os dados recebidos. Evidência no Anexo C, Figura 14 e 15.
Status	Aprovado.

Teste 4. Alerta ESP32 offline	
Objetivo	Verificar se quando a ESP32 está offline o alerta é gerado corretamente.
Passos	1. Iniciar a simulação da ESP32. 2. Ocupar algum assento. 3. Parar o envio de dados (parar a simulação).
Resultado esperado	1. O Node-RED deve exibir o alerta dentro do card e uma notificação de alerta informando que a ESP32 está offline e há um assento ocupado.
Resultado obtido	O Node-RED exibiu a notificação e o alerta dentro do card. Evidência no Anexo C, Figura 16.
Status	Aprovado.

Teste 5. Alerta de encerramento de rota	
Objetivo	Verificar se quando o motorista tentar encerrar uma rota com um assento ocupado um alerta é emitido.
Passos	1. Iniciar a simulação da ESP32. 2. Iniciar uma rota no aplicativo Android. 3. Ocupar algum assento. 4. Encerrar a rota
Resultado esperado	1. O aplicativo deve exibir um alerta visual ao 1. O Node-RED deve exibir o alerta dentro do card e uma notificação de alerta informando que a ESP32 está offline e há um assento ocupado.
Resultado obtido	O Node-RED exibiu a notificação e o alerta dentro do card. Evidência no Anexo C, Figura 17 e 18.
Status	Aprovado.

6. Conclusão

O projeto VanGuard Kids atingiu seu objetivo principal ao propor e implementar uma solução viável para a prevenção do esquecimento de crianças em transportes escolares. O desenvolvimento do protótipo integrou com sucesso os quatro pilares fundamentais do curso: a estruturação de hardware utilizando o sistema operacional FreeRTOS no microcontrolador ESP32; o uso de protocolos de conectividade, como Wi-Fi e MQTT; a criação de uma plataforma visual de monitoramento e histórico no Node-RED; e o desenvolvimento nativo de um aplicativo Android

reativo, utilizando as melhores práticas da arquitetura MVVM e comunicação direta com o hardware via AIDL.

Como limitação do protótipo atual, destaca-se a simulação dos sensores de peso por meio de potenciômetros. Esse mecanismo foi suficiente para validar o fluxo lógico do sistema, aquisição de dados, publicação via MQTT e atualização em tempo real do dashboard e do aplicativo, mas não valida a capacidade real de discriminação de presença. Não há evidências no escopo atual de que o limiar de peso consiga distinguir de forma confiável uma criança pequena ou adormecida de objetos pesados esquecidos no banco, por exemplo. A aplicação em assentos reais exigiria não apenas um processo rigoroso de calibração, mas possivelmente a adoção de sensoriamento complementar (combinando a medição de peso a variáveis como presença térmica ou movimento).

Para melhorias futuras, sugere-se a adoção de sensores de pressão reais e a investigação de uma integração direta com as ECUs (*Engine Control Units*) do veículo por meio do barramento CAN (*Controller Area Network*). Essa evolução alteraria a arquitetura de aquisição: a coleta de dados dos assentos deixaria de ser sem fio (como ocorre hoje entre o ESP32 e a Raspberry Pi 5), passando a trafegar de forma nativa e cabeada pela rede do veículo. Com isso, o protocolo MQTT seria empregado exclusivamente pela RPi 5 para comunicação externa (dashboards ou nuvem), aumentando a robustez do sistema local. Além disso, a conexão com as ECUs permitiria ações ativas de segurança física, como impedir o trancamento das portas ou acionar alarmes caso haja ocupação ao final da rota. Reconhece-se, no entanto, que essa integração representa um salto de complexidade que depende de acesso a sinais proprietários automotivos e envolve severas implicações de responsabilidade civil, exigindo validação regulatória antes de qualquer aplicação real.

Dessa forma, como o sistema processará dados de presença e localização de menores, categorizados como dados sensíveis pela Lei Geral de Proteção de Dados (LGPD), essa expansão tecnológica exigirá o estabelecimento de políticas rigorosas de retenção, anonimização e controle de acesso antes de sua implementação em um ambiente de produção.

7. Referências

- [1] Espressif. How To Write a Wi-Fi Application. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-guides/wifi.html>. 2025.
- [2] Espressif. ESP-MQTT Documentation. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/protocols/mqtt.html>. 2025.
- [3] Espressif. Analog to Digital Converter (ADC) Oneshot Mode Driver. Disponível em: https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/peripherals/adc_oneshot.html. 2025.
- [4] Eclipse Foundation. Eclipse Paho Android Service. Disponível em: <https://eclipse.dev/paho/files/android-javadoc/index.html>. 2022.
- [5] Google. Android Developers. Fragments. Disponível em: <https://developer.android.com/guide/fragments>. 2026.
- [6] Google. Android Developers. Corrotinas do Kotlin no Android. Disponível em: <https://developer.android.com/kotlin/coroutines?hl=pt-br>. 2026.
- [7] Google. Android Developers. Visão geral do ViewModel. Disponível em: <https://developer.android.com/topic/libraries/architecture/viewmodel?hl=pt-br>. 2026.

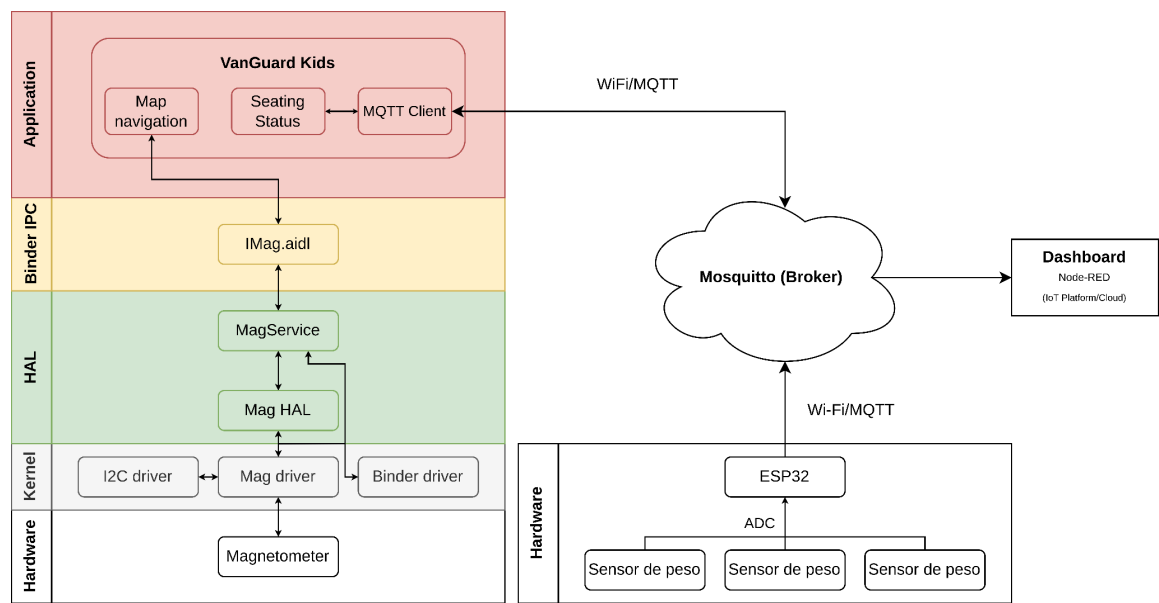
- [8] Google. Android Developers. Camada de Dados. Disponível em: <https://developer.android.com/topic/architecture/data-layer?hl=pt-br>. 2026.
- [9] G1. Menino de 5 anos é esquecido dentro de van escolar e escapa sozinho em Guarulhos. Disponível em: <https://g1.globo.com/sp/sao-paulo/noticia/2026/04/15/menino-de-5-anos-e-esquecido-dentro-de-van-escolar-e-escapa-sozinho-em-guarulhos.ghtml>. 2026.
- [10] G1. Menino de 2 anos morre após ser esquecido dentro de carro por dona de creche, diz polícia. Disponível em: <https://g1.globo.com/go/goias/noticia/2025/02/19/menino-morre-apos-ser-esquecido-dentro-de-carro-pode-dona-de-creche-diz-policia.ghtml>. 2026.
- [11] G1. Corpo de menino que morreu ao ser esquecido em van escolar é enterrado na Zona Leste de SP. Disponível em: <https://g1.globo.com/sp/sao-paulo/noticia/2023/11/16/corpo-de-menino-que-morreu-ao-ser-esquecido-em-van-escolar-e-velado-na-zona-leste-de-sp.ghtml>. 2026.
- [12] Espressif. System Time. Disponível em: https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/system/system_time.html. 2025.
- [13] Wokwi. ESP32 Simulation. Disponível em: <https://docs.wokwi.com/guides/esp32>. 2026.
- [14] Wokwi. Wokwi for VS Code. Disponível em: <https://docs.wokwi.com/vscode/getting-started>. 2026.
- [15] Genymobile. scrpy. Disponível em: <https://github.com/genymobile/scrpy>. 2026.

Anexos

Use esta seção para inserir vídeos (links), imagens, capturas de tela e diagramas que apoiam o relatório. Referencie cada anexo no corpo do texto (ex.: "ver Anexo A").

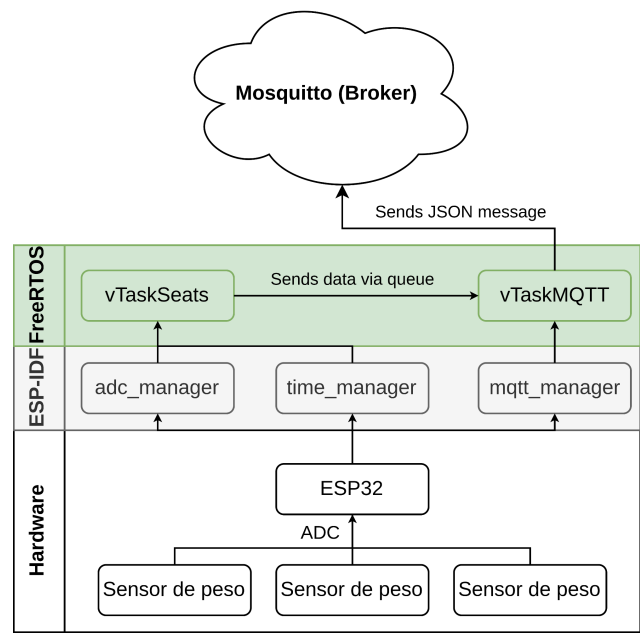
Anexo A — Diagramas de Arquitetura

Figura 1 – Arquitetura do Sistema.



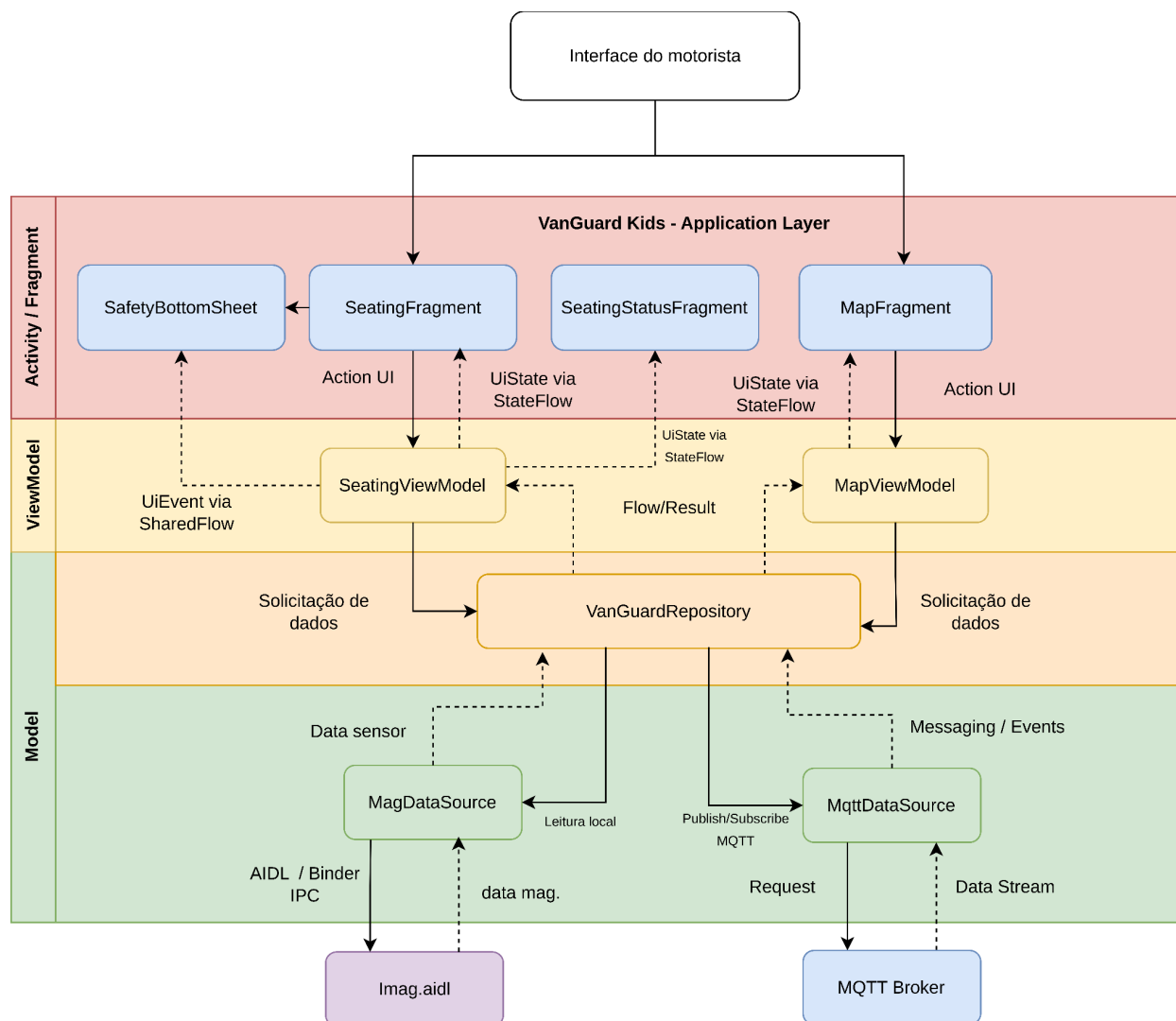
Fonte: elaborado pelo autor (2026).

Figura 2 – Arquitetura do Firmware.



Fonte: elaborado pelo autor (2026).

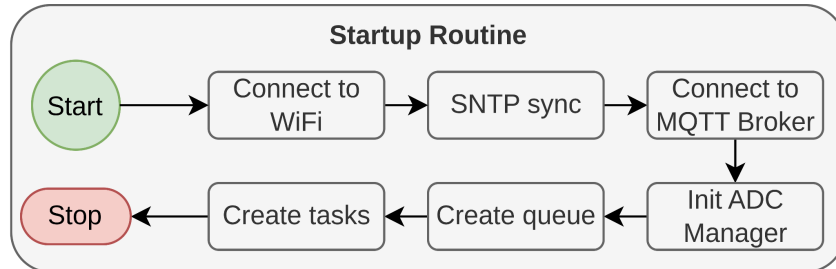
Figura 3 – Arquitetura da aplicação.



Fonte: elaborado pelo autor (2026).

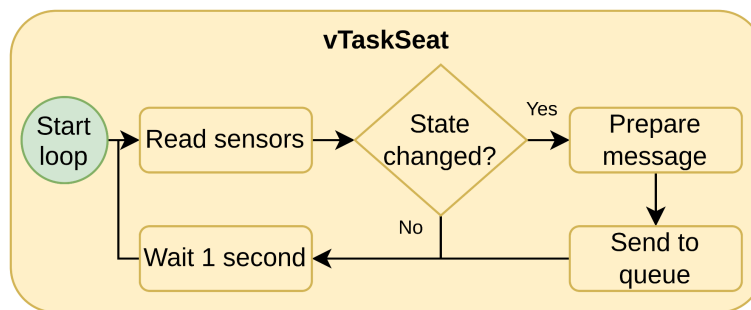
Anexo B — Fluxogramas

Figura 4 – Rotina de inicialização do firmware.



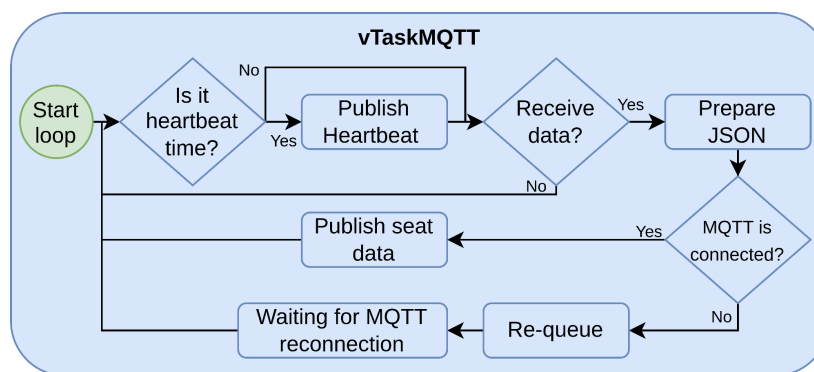
Fonte: elaborado pelo autor (2026).

Figura 5 – Fluxograma da task de leitura dos assentos.



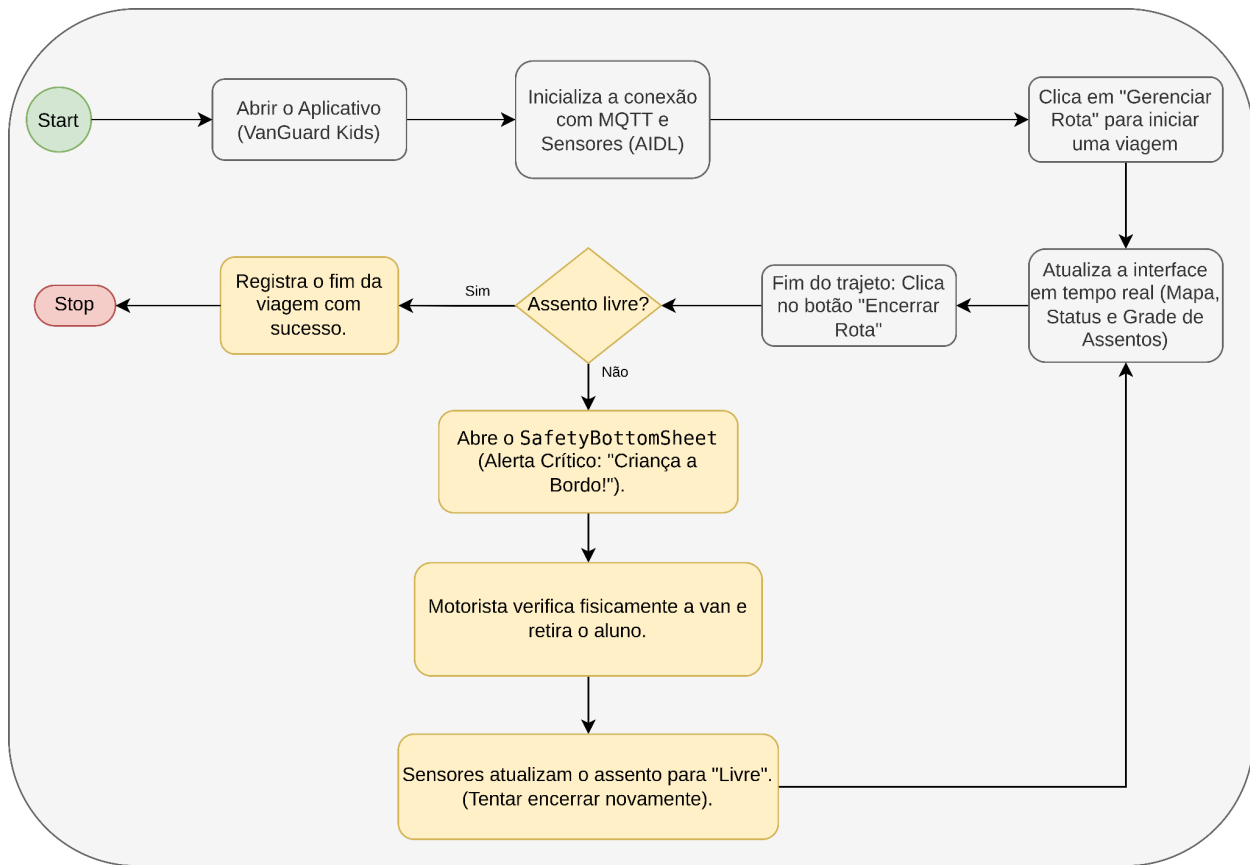
Fonte: elaborado pelo autor (2026).

Figura 6 – Fluxograma da task de envio de mensagens MQTT.



Fonte: elaborado pelo autor (2026).

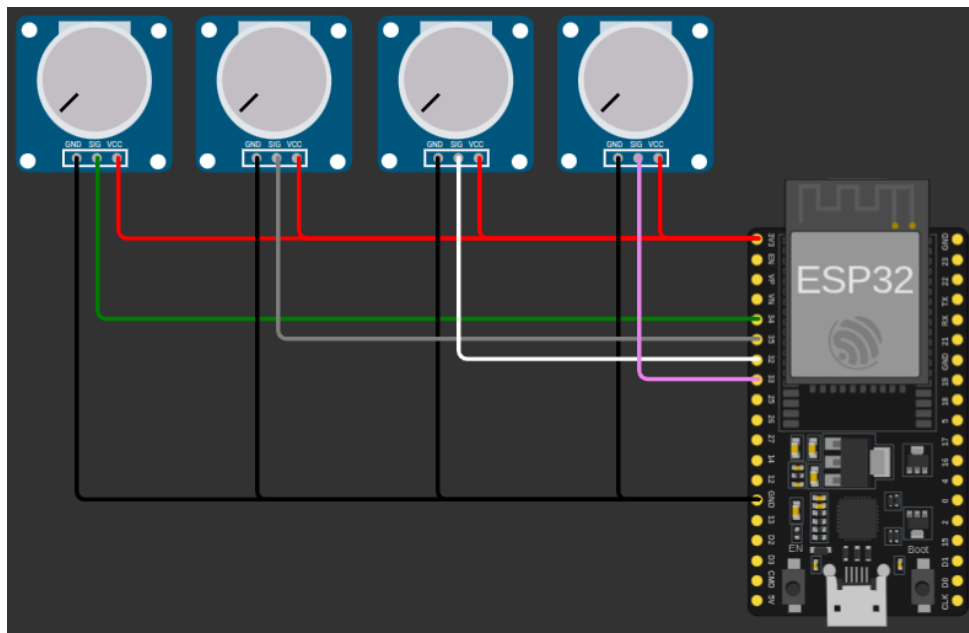
Figura 7 – Fluxo do usuário.



Fonte: elaborado pelo autor (2026).

Anexo C — Protótipo

Figura 8 – Circuito elétrico do protótipo microcontrolado.



Fonte: elaborado pelo autor (2026).

Figura 9 – Dados do assentos

```
samuel in ~
+ mosquitto_sub -h localhost -p 1883 -u vanguard_kids -P vanguard_kids_pw -t vanguard-kids/seats/+
{"client_id":"vanguard-esp32","seat":"seat-01","adc_raw":773,"voltage":0.62,"weight_g":9438,"occupied":true,"ts":1783885949}
{"client_id":"vanguard-esp32","seat":"seat-02","adc_raw":789,"voltage":0.64,"weight_g":9634,"occupied":true,"ts":1783885949}
{"client_id":"vanguard-esp32","seat":"seat-03","adc_raw":757,"voltage":0.61,"weight_g":9243,"occupied":true,"ts":1783885949}
{"client_id":"vanguard-esp32","seat":"seat-04","adc_raw":865,"voltage":0.70,"weight_g":10562,"occupied":true,"ts":1783885949}
{"client_id":"vanguard-esp32","seat":"seat-01","adc_raw":0,"voltage":0.00,"weight_g":0,"occupied":false,"ts":1783886003}
{"client_id":"vanguard-esp32","seat":"seat-02","adc_raw":0,"voltage":0.00,"weight_g":0,"occupied":false,"ts":1783886004}
{"client_id":"vanguard-esp32","seat":"seat-03","adc_raw":0,"voltage":0.00,"weight_g":0,"occupied":false,"ts":1783886005}
{"client_id":"vanguard-esp32","seat":"seat-04","adc_raw":0,"voltage":0.00,"weight_g":0,"occupied":false,"ts":1783886007}
```

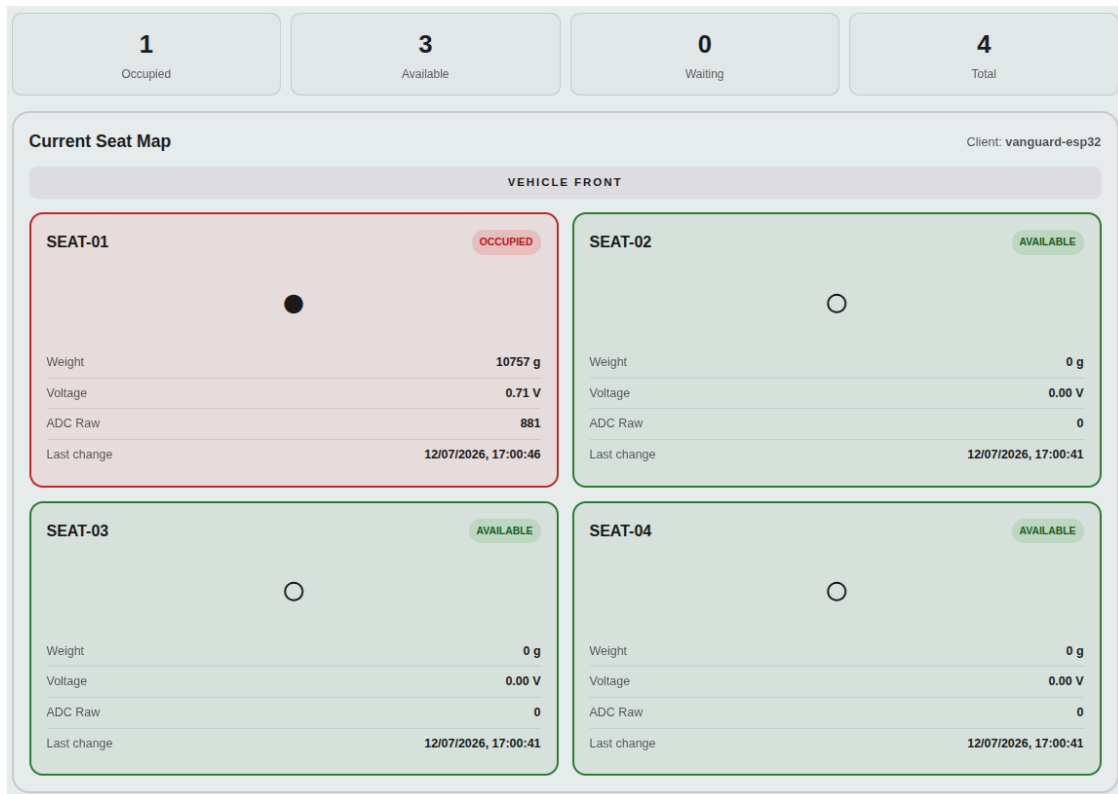
Fonte: elaborado pelo autor (2026).

Figura 10 – Dados de heartbeat

```
samuel in ~
+ mosquitto_sub -h localhost -p 1883 -u vanguard_kids -P vanguard_kids_pw -t vanguard-kids/heartbeat
{"client_id":"vanguard-esp32","ts":1783885949,"seats":4}
{"client_id":"vanguard-esp32","ts":1783885979,"seats":4}
{"client_id":"vanguard-esp32","ts":1783886009,"seats":4}
```

Fonte: elaborado pelo autor (2026).

Figura 11 – Mapa de assentos no Node-RED



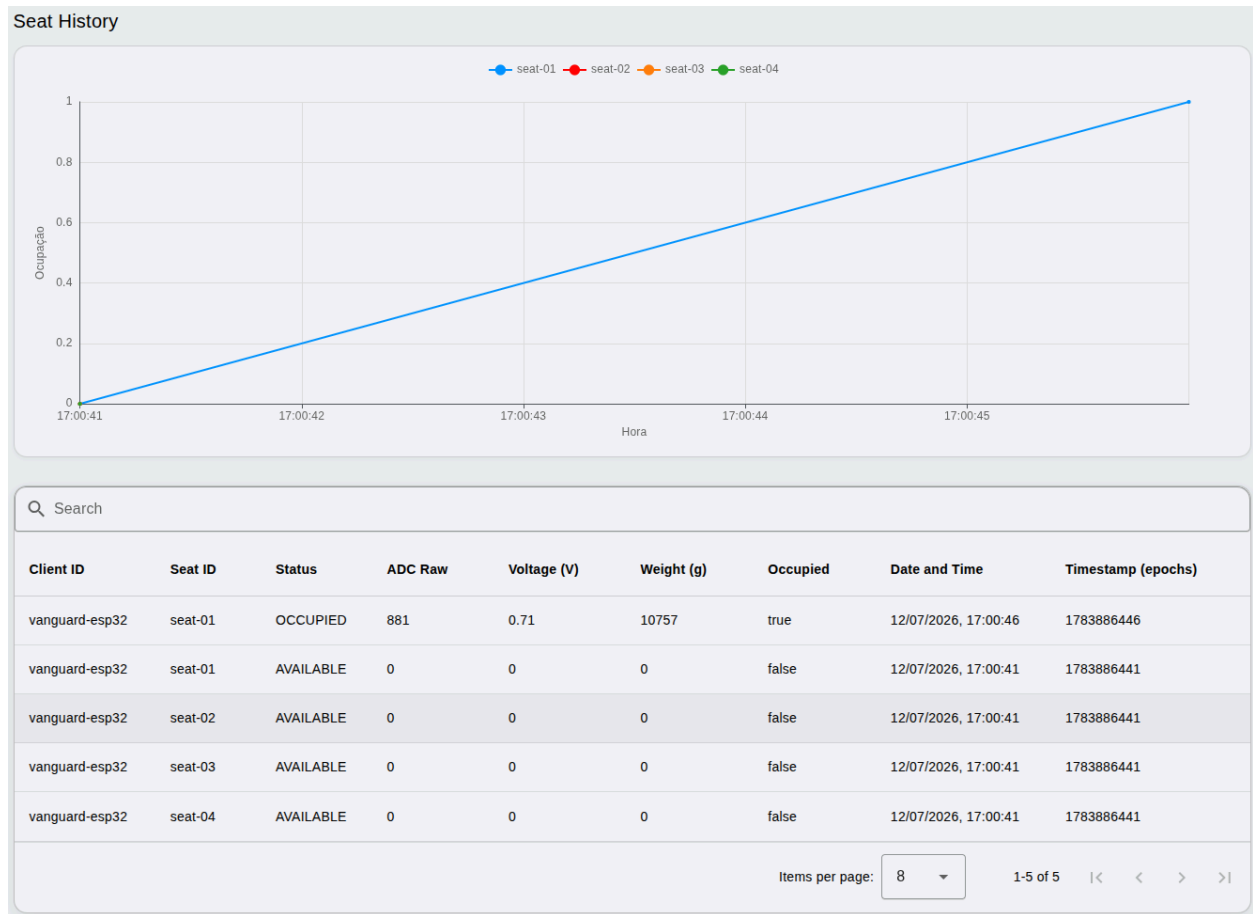
Fonte: elaborado pelo autor (2026).

Figura 12 – Card para heartbeat no Node-RED



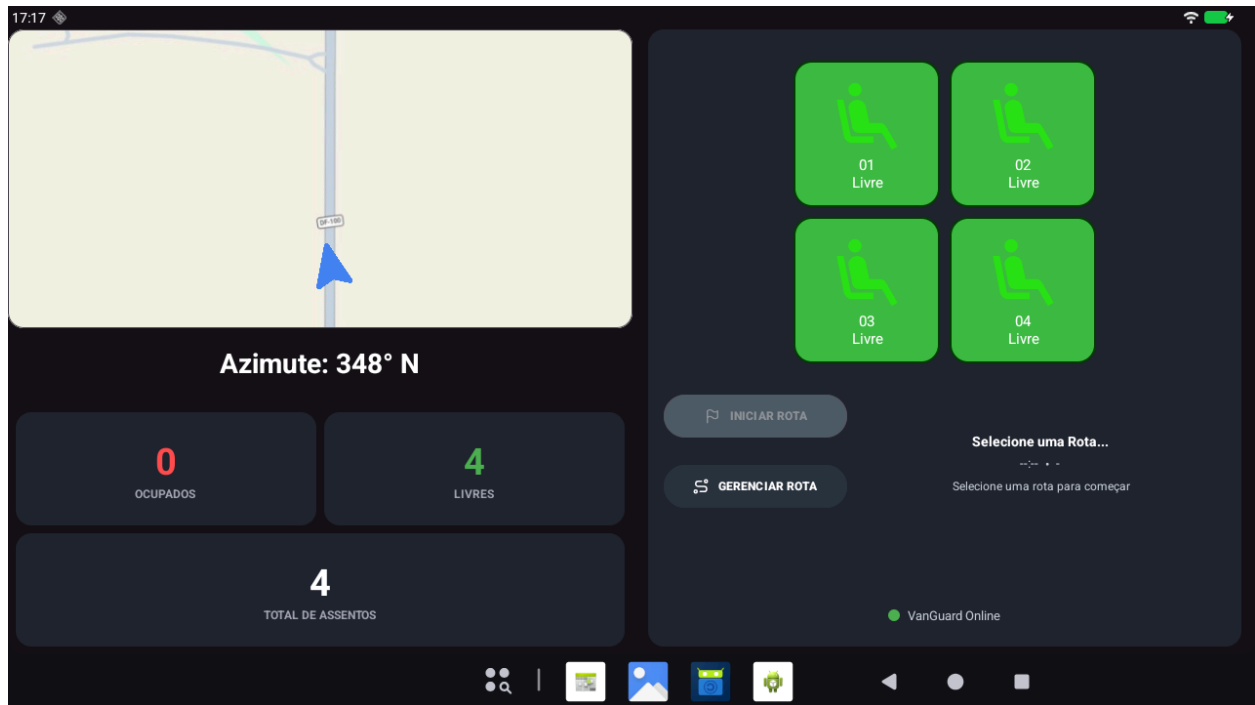
Fonte: elaborado pelo autor (2026).

Figura 13 – Histórico de assentos no Node-RED



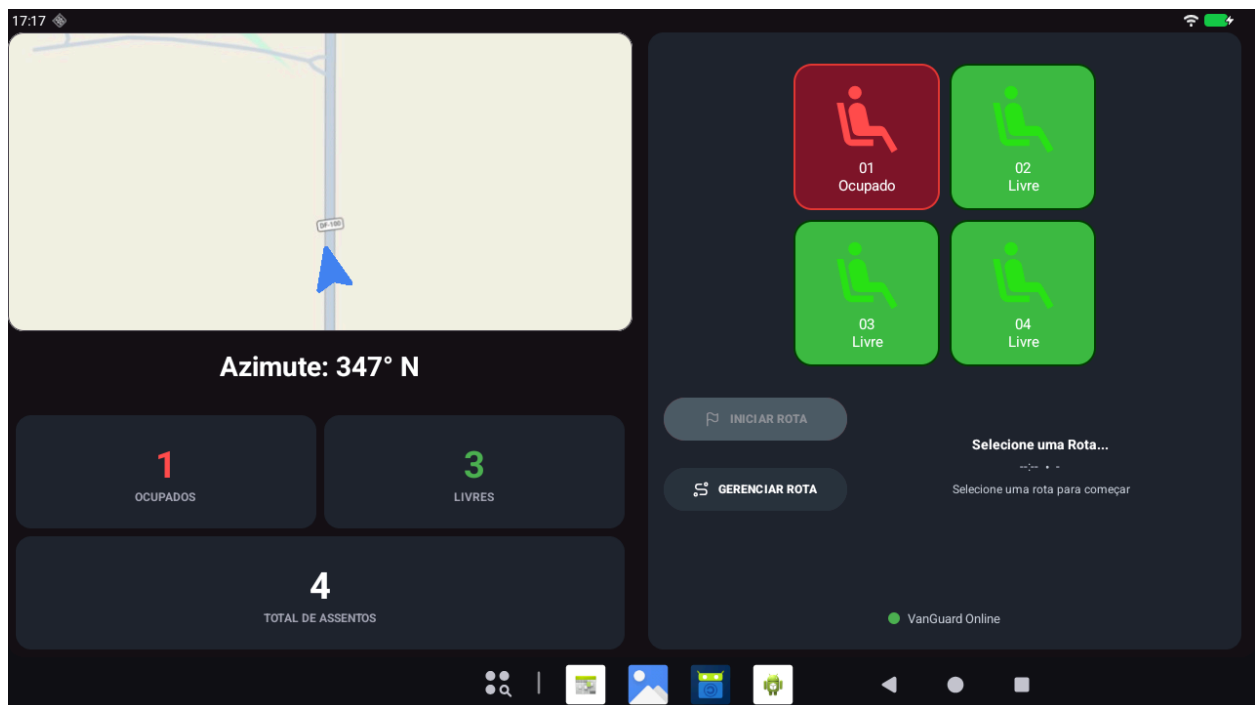
Fonte: elaborado pelo autor (2026).

Figura 14 – Interface do aplicativo com todos os assentos livres



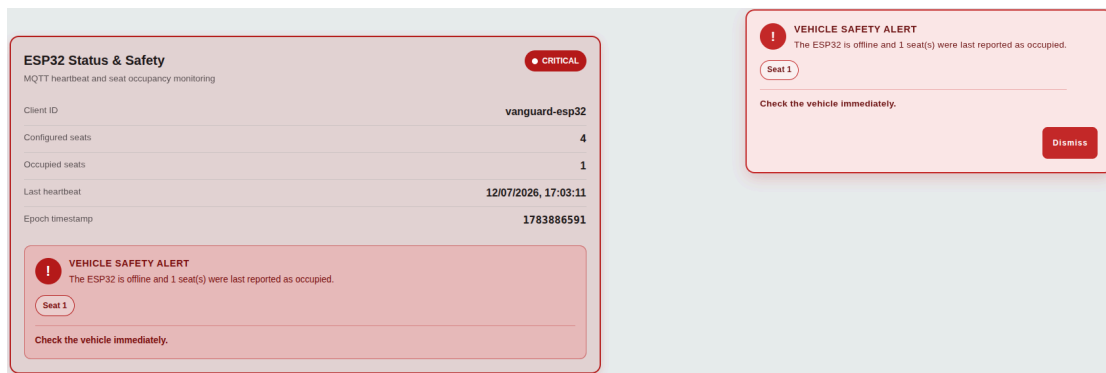
Fonte: elaborado pelo autor (2026).

Figura 15 – Interface do aplicativo com um assento ocupado



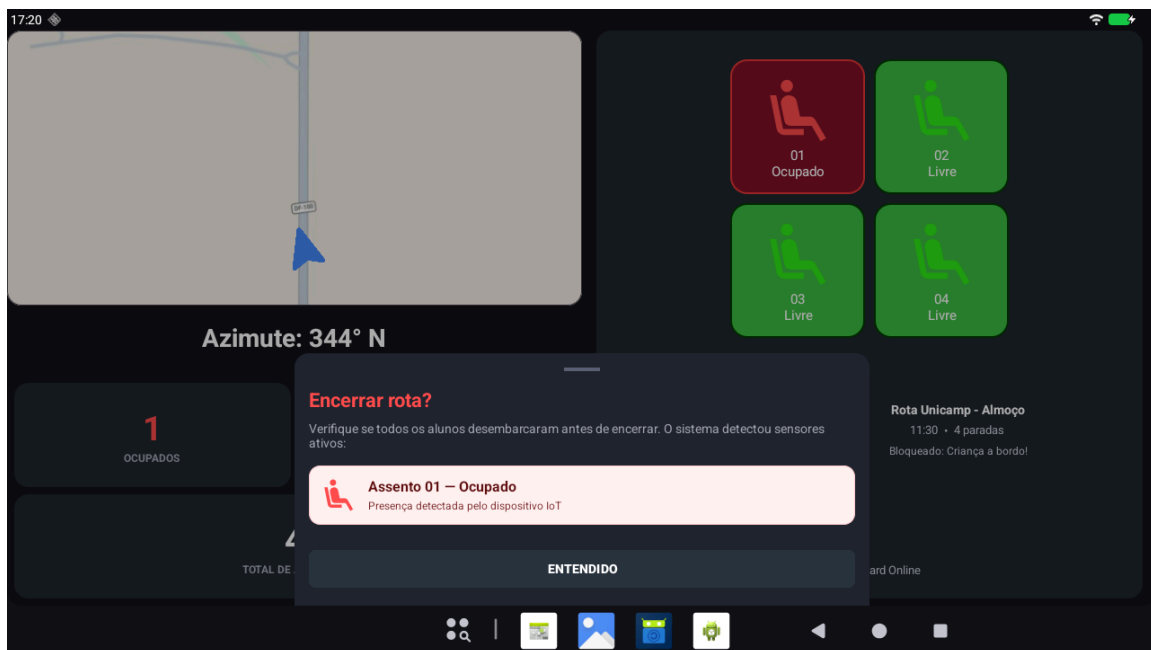
Fonte: elaborado pelo autor (2026).

Figura 16 – Alerta de heartbeat no Node-RED



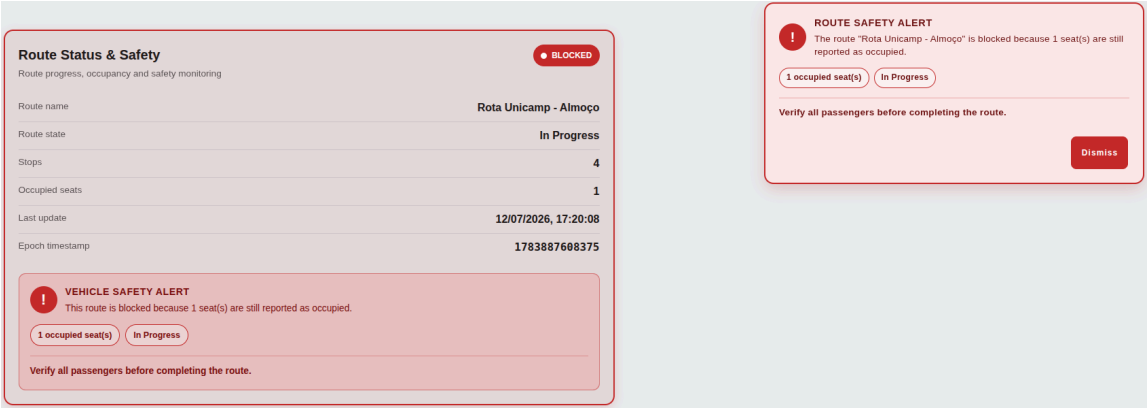
Fonte: elaborado pelo autor (2026).

Figura 17 – Alerta de encerramento de rota com uma poltrona ocupada



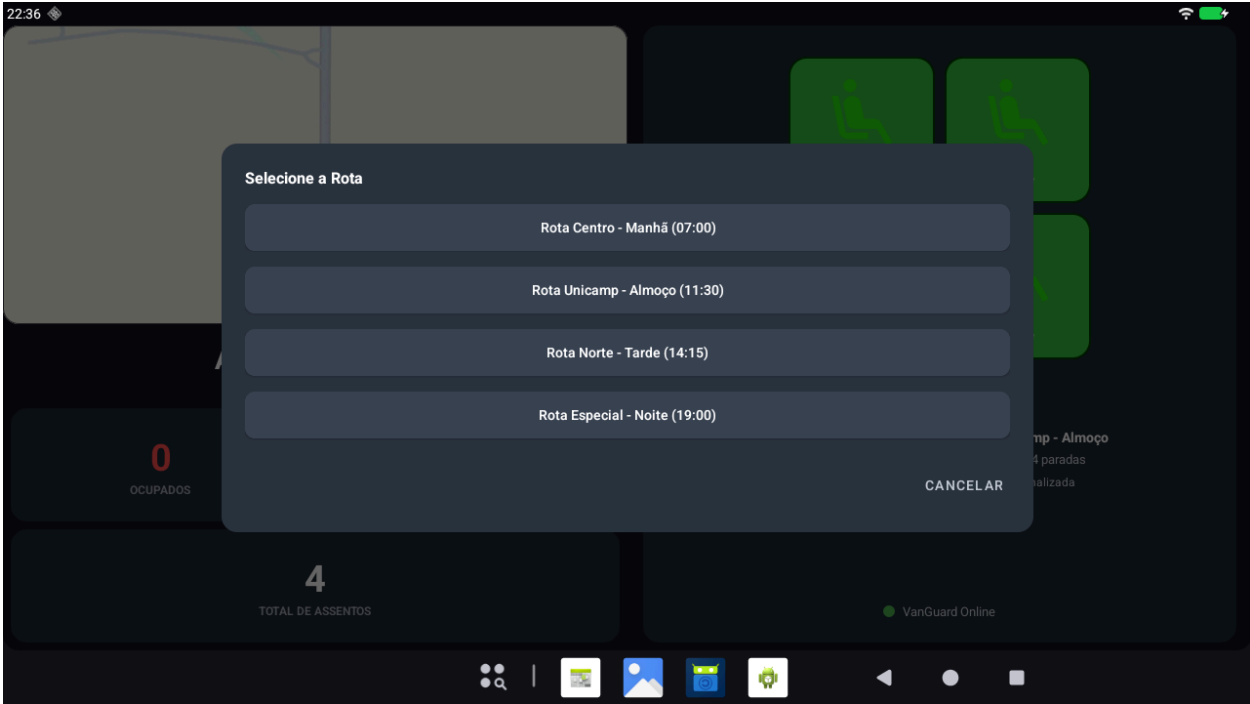
Fonte: elaborado pelo autor (2026).

Figura 18 – Alerta de encerramento de roda no Node-RED



Fonte: elaborado pelo autor (2026).

Figura 19 – Interface para selecionar uma rota



Fonte: elaborado pelo autor (2026).

Anexo D — Vídeos de Demonstração

Tabela 4 – Link para vídeo de demonstração

Vídeo 1 — Demonstração	Título: Capacitação em IoT & Android Embarcado Projeto Final VanGuard Kids. Link: https://youtu.be/3pfaNkhZVWY Duração: 12:06 minutos.
------------------------	---

Anexo E — Material Complementar

Tabela 5 – Reportagens

Reportagem 1	Título: Menino de 5 anos é esquecido dentro de van escolar e escapa sozinho em Guarulhos. Link: https://g1.globo.com/sp/sao-paulo/noticia/2026/04/15/menino-de-5-anos-e-esquecido-dentro-de-van-escolar-e-escapa-sozinho-em-guarulhos.ghtml
Reportagem 2	Título: Menino de 2 anos morre após ser esquecido dentro de carro por dona de creche, diz polícia. Link: https://g1.globo.com/go/goias/noticia/2025/02/19/menino-morre-apos-ser-esquecido-dentro-de-carro-pode-dona-de-creche-diz-policia.ghtml
Reportagem 3	Título: Corpo de menino que morreu ao ser esquecido em van escolar é enterrado na Zona Leste de SP. Link: https://g1.globo.com/sp/sao-paulo/noticia/2023/11/16/corpo-de-menino-que-morreu-ao-ser-esquecido-em-van-escolar-e-velado-na-zona-leste-de-sp.ghtml